

National Textile University, Faisalabad



Name:	M. Saad Ehtsham
Class:	BS-CS(B)
Registration No:	23-NTU-CS-1072
Assignment:	02
Course Name:	IOT & Embedded system
Submitted To:	Sir Nasir

Question-1 ESP32 Webserver (webserver.cpp)

Part A: Short Questions

1. What is the purpose of Webserver server (80), and what does port 80 represent?

The Webserver server (80) is used to create a web server object on the ESP32. This allows the ESP32 to act like a small web server that can send web pages to users through a browser.

Port 80 represents the default HTTP port. When a user enters the ESP32's IP address in a web browser without specifying a port number, the browser automatically tries to connect using port 80. Therefore, using port 80 makes the ESP32 web server easy to access using any standard web browser.

2. Explain the role of `server.on("/", handleRoot)` in this program.

The statement `server.on("/", handleRoot);` is used to define how the ESP32 should respond when a user accesses the root URL ("/") of the web server.

- "/" represents the home page of the web server.
- `handleRoot` is a function that is executed when the root page is requested.

So, this line connects the web address (home page) to the function that generates and sends the web content to the user.

3. Why is `server.handleClient();` placed inside the `loop()` function? What will happen if it is removed?

`server.handleClient();` is placed inside the `loop()` function so that the ESP32 can continuously check for incoming client requests, such as a browser trying to access the web page.

The `loop()` function runs repeatedly that allows the server to respond instantly whenever a request is received.

If it is removed:

- The ESP32 will not listen for client requests.
- The web page will not load in the browser.
- The server will appear unresponsive.

So, this line is essential for keeping the web server active.

4. In `handleRoot()`, explain the statement: `server.send(200, "text/html", html);`

This is used to send a response from the ESP32 web server to the client's browser.

- 200 is the HTTP status code, which means request successful.
- "text/html" tells the browser that the content being sent is an HTML web page.
- html is the variable that contains the actual HTML code to be displayed.

Together, this line sends the web page content to the browser so the user can see it.

5. What is the difference between displaying last measured sensor values and taking a fresh DHT reading inside `handleRoot()`?

Displaying last measured sensor values means the ESP32 shows previously stored sensor data without reading the sensor again. This approach is faster and reduces sensor usage.

Taking a fresh DHT reading inside `handleRoot()` means the sensor is read every time a user refreshes the web page. While this provides updated values, it can cause:

- Slower page loading
- Unstable readings
- Possible sensor errors (especially with DHT sensors)

Part B: Long Question

Describe the complete working of the ESP32 webserver-based temperature and humidity monitoring system.

1. ESP32 Wi-Fi Connection and IP Address Assignment

When the ESP32 starts, it connects to a Wi-Fi network using the given SSID and password. Once connected, the router assigns an IP address to the ESP32, which is used to open the web page in a browser.

```
WiFi.begin(ssid, password);
while (WiFi.status() != WL_CONNECTED) {
  delay(500);
}
Serial.println(WiFi.localIP());
```

2. Web Server Initialization and Request Handling

After connecting to Wi-Fi, the ESP32 starts a web server on port 80. The server is programmed to respond when the root URL is accessed.

```
WebServer server(80);  
  
server.on("/", handleRoot);  
server.begin();
```

3. Button-Based Sensor Reading and OLED Update

A push button is used to control when the DHT sensor is read. When the button is pressed, the ESP32 takes temperature and humidity readings and updates the OLED display.

```
if (digitalRead(buttonPin) == LOW) {  
    temperature = dht.readTemperature();  
    humidity = dht.readHumidity();  
}
```

4. Dynamic HTML Webpage Generation

The ESP32 generates an HTML page dynamically by inserting sensor values into an HTML string before sending it to the browser.

```
String html = "<html><body>";  
html += "<h2>ESP32 Temperature & Humidity</h2>";  
html += "<p>Temperature: " + String(temperature) + " C</p>";  
html += "<p>Humidity: " + String(humidity) + " %</p>";  
html += "</body></html>";
```

This HTML content is sent to the browser as a web page.

5. Purpose of Meta Refresh in Webpage

Meta refresh is used to automatically reload the web page so the sensor values stay updated without manual refreshing.

```
<meta http-equiv="refresh" content="5">
```

6. Common Issues and Their Solutions

- One common issue is Wi-Fi connection failure, which usually happens due to incorrect credentials. This can be solved by rechecking the SSID and password.
- Another issue is slow or frozen web pages, which occurs if `server.handleClient()` is missing from the loop. Adding it back fixes the problem.
- DHT sensor errors can occur if readings are taken too frequently. Using a button or adding delays helps avoid this issue.
- OLED display problems are usually caused by wrong I2C address or loose connections, which can be fixed by checking wiring and display address.

Question-2 Blynk Cloud Interfacing (blynk.cpp)

Part-A: Short Questions

1. What is the role of Blynk Template ID in an ESP32 IoT project? Why must it match the cloud template?

The Blynk Template ID is used to connect the ESP32 project with a specific template created on the Blynk Cloud. This template contains the widgets, layout, and virtual pins that we design in the Blynk app.

It must match the cloud template because if the ID is wrong, the ESP32 will not know which project it belongs to. As a result, the device will fail to connect and data will not appear on the Blynk dashboard.

2. Differentiate between Blynk Template ID and Blynk Auth Token.

The Blynk Template ID is used to identify the project or template on the Blynk Cloud. It tells the system which dashboard design and widgets should be used.

On the other hand, the Blynk Auth Token is a unique key that identifies the specific hardware device. It is used for authentication so that Blynk knows which ESP32 is sending or receiving data.

3. Why does using DHT22 code with a DHT11 sensor produce incorrect readings? Mention one key difference between the two sensors.

Using DHT22 code with a DHT11 sensor gives wrong readings because both sensors work differently and have different data formats. The ESP32 expects DHT22-type data, but the DHT11 sends data in a simpler and limited form.

- One key difference is that the DHT22 has higher accuracy and a wider temperature range, while the DHT11 is less accurate and has a smaller range.

4. What are Virtual Pins in Blynk? Why are they preferred over physical GPIO pins for cloud communication?

Virtual Pins in Blynk are software-based pins that are used to send and receive data between the ESP32 and the Blynk Cloud. They do not belong to any physical pin on the ESP32.

They are preferred because cloud communication does not depend on actual hardware pins. Virtual pins make the system more flexible, easy to manage, and safer for sending sensor data and control signals through the internet.

5. What is the purpose of using BlynkTimer instead of delay() in ESP32 IoT applications?

Blynk Timer is used to run tasks at specific time intervals without stopping the program. It allows the ESP32 to do multiple things at the same time, such as reading sensors and communicating with the cloud.

Using delay() stops the whole program, which can disconnect the ESP32 from Blynk and cause slow performance. Therefore, Blynk Timer is preferred because it keeps the system responsive and stable.

Part-B: Long Question

Creation of Blynk Template and Datastreams

First, a template is created on the Blynk Cloud. This template is used to design how the data will be shown on the Blynk app. Datastreams are added inside the template using virtual pins, usually one for temperature and one for humidity.

Create New Template

NAME
DHT 3 / 50

HARDWARE
ESP32

CONNECTION TYPE
Wifi

DESCRIPTION
Description 0 / 128

Cancel Done

 SAADs DHT Edit

Datastreams

Search datastream

ID	Name	Pin	Color	Data Type	Units	Is Raw	Min	Max	Decimals
1	Temperture	V0		Double	°C	false	0	60	###
2	Humidity	V1		Double	%	false	0	100	###

Role of Template ID, Template Name, and Auth Token

The Template ID is used to link the ESP32 code with the correct Blynk project. The Template Name helps in identifying the project on the Blynk platform. The Auth Token is a unique key that connects the ESP32 device to the Blynk Cloud. If any of these are incorrect, the device will not connect.

```
#define BLYNK_TEMPLATE_ID "TMPL6KM7Lch7W"  
#define BLYNK_TEMPLATE_NAME "SAADs DHT"  
#define BLYNK_AUTH_TOKEN "G8LkZV9Qp_ZWCswos1yYo3j7iPvtlrfb"
```

Sensor Configuration (DHT11 vs DHT22)

A DHT sensor is used to measure temperature and humidity. DHT11 and DHT22 are different sensors, so the correct sensor type must be selected in the code. Using the wrong sensor configuration results in incorrect readings.

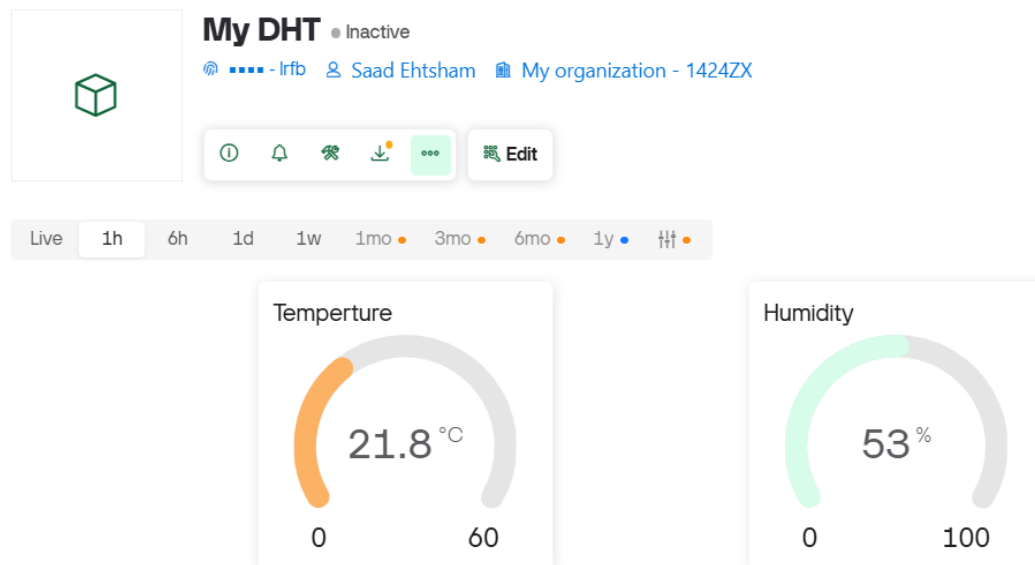
Sending Data to Blynk Cloud

After reading the sensor values, the ESP32 sends temperature and humidity data to the Blynk Cloud using `Blynk.virtualWrite()`. Virtual pins are used to transfer data from the ESP32 to the Blynk app.

Common Problems and Their Solutions

Common problems include Wi-Fi connection issues, wrong Template ID or Auth Token, and incorrect sensor selection. These problems can usually be solved by checking network details, Blynk credentials, and sensor configuration.

BLYNK WEB DASHBOARD



MOBILE DASHBOARD:

